

RELATORIO DO PROJETO GERENCIADOR DE MEMÓRIA

Disciplina: Sistema operacionais

Professor: Clovis Ferraro

Grupo: 2

SUMARIO

1-	INTRODUÇÃO	1
2-	METODOLOGIA:	2
2-1	SISTEMAS OPERACIONAIS	2
2-1.1	Linux	2
2-1.2	Windows	4
2-1.3	Android (via ADB):	9
3-	ANÁLISE DOS COMANDOS E CONFIGURAÇÕES	12
3-1	ANALISE DE USO DA MEMÓRIA NO LINUX:	12
3-1.1	Análise de Memória do Processo:	14
3-1.2	Gestão da Memória Virtual:	17
3-2	ANALISE DE USO DA MEMÓRIA NO WINDOWS:	20
3-2.1	Monitoramento da Memória do Sistema:	22
3-2.2	Pagefile (Arquivo de paginação/Swap):.....	26
3-2.3	Análise de memória por processo:	28
3-3	ANDROID:	34
4-	COMPARAÇÃO E ANALISE CRÍTICA:	37
4-1	DISCUSSÃO	38
5.	CONCLUSÃO	39
6.	AUTOAVALIAÇÃO	40
7.	REFERENCIAS:	41

1- INTRODUÇÃO

O principal objetivo desse relatório é analisar e otimizar o uso e a alocação de memória pelo sistema operacional nos ambientes Windows, Linux e Android. Entendendo como cada sistema gerencia a sua memória, incluindo o uso da cache, alocação dinâmica e também o gerenciamento de memória virtual. E por fim compreender como fazer o uso da memória de forma eficiente. Devido ao fato da memória ser um recurso limitado e crucial para máquina, é muito importante fazer um uso eficiente da mesma. Para o desempenho, ao fazer uma boa administração da memória, os programas tem o acesso aos dados de forma mais rápida e também a possibilidade de executar vários aplicativos simultaneamente, evitando lentidão e a necessidade de fazer o swap. O bom uso da memória é importante para a estabilidade, porque é possível controlar a quantidade de recursos que os processos usam, evitando assim o uso de espaço desnecessário. O gerenciamento da memória de forma adequada, é também muito importante quando se trata da segurança, pois evita que ocorra invasões, como o buffer overflow, que é quando um programa tenta inserir mais dados em um buffer do que ele foi projetado para guardar, ocorrendo um “transbordamento” de dados, sobrescrevendo espaços de memória adjacentes. E por fim para os dispositivos móveis, é essencial ter um gerenciamento eficiente, por causa dos recursos limitados, como o processamento e a bateria.

2- METODOLOGIA:

2-1 Sistemas Operacionais

Linux — Ubuntu 22.04 LTS: ambiente de testes executado em máquina virtual. Testes realizados via terminal (shell)

Windows — Windows 11: testes executados em máquina física. Ferramentas gráficas e PowerShell utilizadas.

Android — Android 13 (via ADB Shell): dispositivo físico conectado por USB com Depuração USB ativada; testes via adb shell

2-1.1 *Linux*:

Para o ambiente Linux, foi realizada uma análise do consumo e gerenciamento de memória utilizando ferramentas nativas de linha de comando, que permitem observar o comportamento do sistema em tempo real. Os principais comandos empregados foram `free -h`, `swapon --show` e `vmstat`, além de utilitários complementares como `top`, `htop` e `cat /proc/meminfo`, que fornecem informações de baixo nível diretamente do kernel. `free -h`

O comando `free -h` exibe um resumo geral da utilização da memória RAM e da memória swap do sistema.

O parâmetro `-h` significa “human readable”, apresentando os valores em unidades como MB ou GB.

A saída inclui as colunas total, usada, livre, compartilhada, buffers/cache e disponível.

Esse comando permite avaliar rapidamente quanto da memória física está sendo consumida e qual parcela está em cache ou disponível para novos processos.

Durante o teste, observou-se que o sistema Linux aloca parte da RAM para o cache de disco, o que melhora o desempenho, mas pode ser liberado automaticamente quando novos processos exigem mais memória.

`swapon --show`

O comando `swapon --show` foi utilizado para verificar as partições ou arquivos de swap ativos.

A memória swap é um espaço no disco rígido usado como extensão da RAM, permitindo que o sistema continue operando mesmo quando a memória física está quase cheia.

Na análise, foram exibidos campos como:

1. NAME (nome do dispositivo de swap),
2. TYPE (tipo de partição ou arquivo),

3. SIZE (tamanho total),
4. USED (quantidade atualmente em uso),
5. PRIO (prioridade de uso).

Essas informações permitem confirmar se o swap está configurado corretamente e se está sendo utilizado para aliviar a pressão sobre a RAM.

vmstat

O comando vmstat (Virtual Memory Statistics) fornece uma visão dinâmica dos recursos de memória, processos, sistema de paginação e CPU.

Sua execução mostra métricas como: r

(número de processos prontos para execução),

b (processos bloqueados), swpd

(memória usada pelo swap), free, buff,

cache (memória livre,

buffers e cache), si/so (entrada e saída de páginas

de swap), us, sy, id, wa (percentuais de uso de

CPU).

A análise do vmstat permite observar o comportamento dinâmico da memória virtual e identificar situações de paginação excessiva, que podem indicar gargalos ou vazamentos de memória.

Resultados observados

Durante a execução dos testes, o Linux apresentou uso eficiente de memória, com grande parte do espaço “usado” sendo destinado a cache e buffers — uma característica típica do sistema, que reutiliza memória ociosa para otimizar o acesso a arquivos e programas.

O swap foi ativado, mas com baixo uso, indicando que o sistema possuía memória RAM suficiente para as tarefas em andamento.

Os comandos vmstat e top mostraram baixa taxa de paginação e estabilidade nos processos, sugerindo ausência de vazamentos e bom controle de memória virtual.

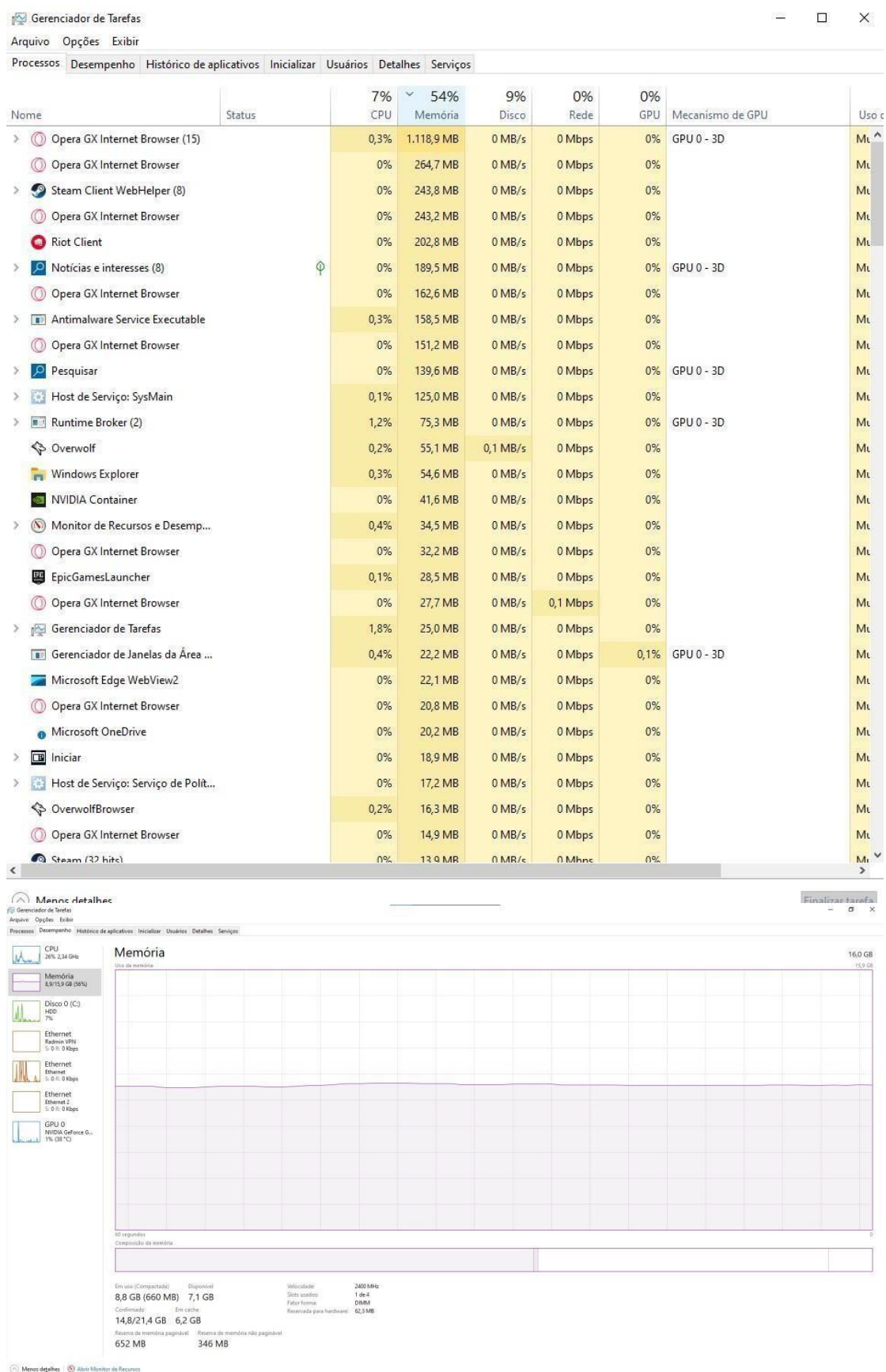
Em conclusão, o gerenciamento de memória no Linux demonstrou ser altamente otimizado, com uso inteligente de cache, paginação controlada e alocação eficiente entre processos e kernel, mantendo o sistema responsivo mesmo sob carga moderada.

2-1.2 Windows

Task Manager (Gerenciador de Tarefas)

- O que faz: interface rápida para ver uso de CPU, memória, disco, rede por processo. Mostra memória em uso por processo (Private Working Set).

- Como usar: Ctrl+Shift+Esc → aba Processos (ordene por Memória). Aba Desempenho mostra total de memória, em uso, disponível, cache, e gráfico.
- processos com uso crescente em Ociosidade indicam vazamento; alto uso de Standby pode ser cache de sistema.



(Task Manager – Processos e Desempenho)

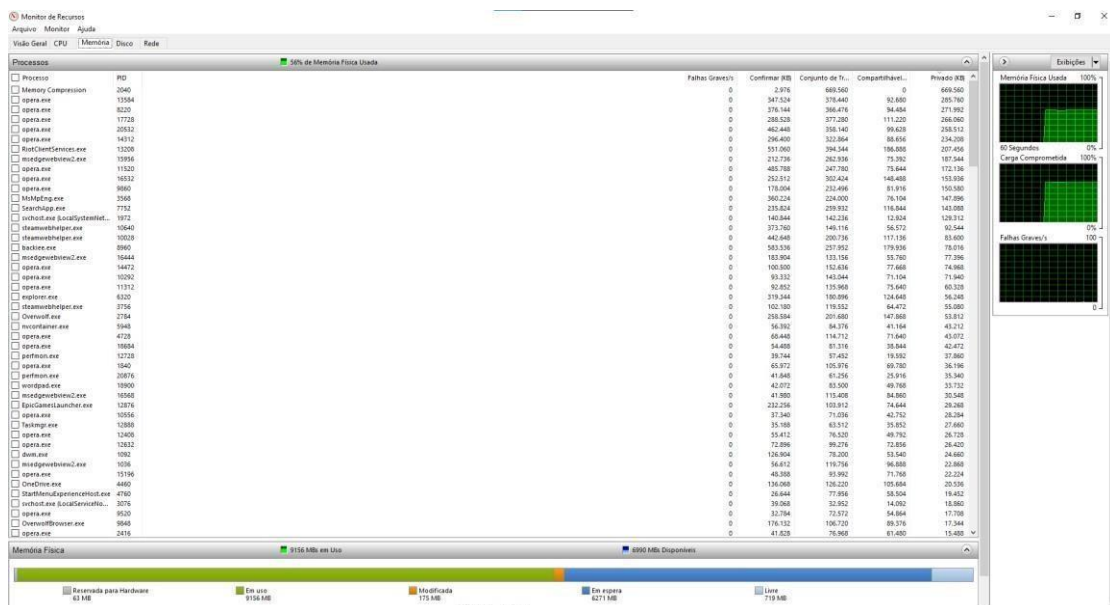
As imagens apresentam o uso de memória em um sistema com 16 GB de RAM. A aba “Desempenho > Memória” indica que aproximadamente 8,8–9,0 GB (~55–56%) da memória física está em uso, com cerca de 7,1 GB disponíveis; a memória compactada é de ~660 MB.

Essas medidas mostram que o sistema opera com folga e não há evidência imediata de degradação por falta de memória.

Na aba Processos, observa-se que o Opera GX é o maior consumidor, aparecendo com múltiplos processos (modelo de multiprocessos do navegador). Um dos processos do Opera consome ~11,1 GB e vários outros consomem entre 150–300 MB cada, o que é consistente com muitas abas, processos de render, extensões e helpers. A soma desses processos explica boa parte do uso total de RAM.

Resource Monitor (resmon)

- O que faz: detalhe de uso de memória por processo, incluindo Commit, Working Set, Shareable, Private.
- Como usar: Win + R → resmon → aba Memory.
- Commit alto significa muita memória virtual reservada; Hard Faults/sec indica paginação para disco.



O Monitor de Recursos do Windows foi utilizado para observar o comportamento da memória física (RAM) em tempo real, possibilitando uma análise detalhada do consumo por processo. Na imagem registrada, o sistema apresentava 56% de memória física utilizada, valor considerado estável e dentro de um uso normal.

Entre os processos em execução, destacam-se diversas instâncias do opera.exe, indicando múltiplas abas ou janelas abertas do navegador, além de processos relacionados a plataformas de jogos, como steamwebhelper.exe e EpicGamesLauncher.exe, que são conhecidos por consumir quantidades significativas de RAM. Também é possível observar o processo Memory Compression, responsável por realizar a compressão de blocos de memória

a fim de otimizar o uso da RAM e evitar o acionamento constante do arquivo de paginação no disco.

No painel inferior, é possível verificar que o sistema possui aproximadamente 16 GB de memória física, dos quais 9.156 MB estão em uso, 6.990 MB disponíveis e apenas 63 MB reservados para o hardware. Isso demonstra que há uma quantidade significativa de memória livre e em cache, o que favorece o desempenho e reduz o tempo de acesso a dados frequentemente utilizados.

Além disso, o campo Falhas Graves apresenta valor igual a zero em todos os processos, indicando que nenhuma falha de página grave ocorreu durante a execução. Isso significa que os dados requisitados pelos programas estavam disponíveis na memória física, sem necessidade de carregamento do disco. um indicativo de boa performance do sistema.

De forma geral, a análise mostra um gerenciamento eficiente de memória, com uso equilibrado entre memória física, compressão e cache, e sem sinais de vazamentos de memória ou paginação excessiva. O sistema está estável e responde bem à carga de trabalho apresentada.

Performance Monitor (perfmon)

- O que faz: ferramenta para adicionar contadores (ex.: \Memory\Available MBytes, \Memory\Pages/sec, \Process(*)\Private Bytes) e coletar histórico e relatórios.
- Como usar: perfmon > Data Collector Sets (User Defined) > criar coleta por X tempo > exportar relatório.
- aumento de Pages/sec correlacionado com degradação de desempenho. Available MBytes muito baixo > risco de OOM.



Monitor de Desempenho do Windows (Perfmon), ferramenta voltada à coleta de métricas detalhadas sobre o comportamento do sistema. O relatório apresentado contém indicadores referentes ao uso da CPU, memória, disco e rede, além de uma visão diagnóstica do desempenho geral.

O resumo do diagnóstico informa que o uso de memória física estava em 57%, valor semelhante ao observado no Monitor de Recursos, reforçando a consistência entre as ferramentas. O relatório também identifica o processo Memory Compression como ativo, o que confirma o uso da técnica de compressão de memória pelo Windows para otimizar a alocação de dados.

Nos resultados de desempenho, observam-se métricas relevantes, como Bytes confirmados em uso, Falhas de página/s e Limite de confirmação. As falhas de página são praticamente inexistentes, o que indica que o sistema não está sobrecarregado e que os dados requisitados estão sendo atendidos diretamente pela RAM, sem necessidade de recorrer ao arquivo de paginação. Da mesma forma, o limite de confirmação não foi atingido, o que significa que há espaço suficiente para novas alocações de memória virtual.

O relatório também apresenta a quantidade de memória disponível, que gira em torno de 7 GB, confirmando que o sistema dispõe de ampla reserva para execução de novas tarefas. Essa disponibilidade é reforçada pela baixa taxa de paginação (páginas por segundo próximas de zero) e pelos baixos tempos de acesso à memória.

Portanto, a análise do Monitor de Desempenho demonstra que o sistema encontra-se em estado normal de operação, sem gargalos de memória, sem vazamentos identificáveis e com gerenciamento eficiente de recursos. A utilização da memória virtual está controlada e a compressão de memória contribui para o equilíbrio do uso físico da RAM. O desempenho geral do sistema é considerado estável e otimizado, adequado tanto para tarefas comuns quanto para aplicações que demandam maior consumo de memória.

2-1.3 Android (via ADB):

`adb shell dumpsys meminfo`

- O que faz: imprime estatísticas detalhadas de memória por processo: PSS, USS, RSS, dalvik-heap, native heap, graphics, private, shared, total.
- Uso típico: `adb shell dumpsys meminfo` (listagem geral) ou `adb shell dumpsys meminfo <package>` (detalhe por app).
- PSS crescente sem liberar após interações > possível vazamento; alta Native sem relação com uso gráfico > problema nativo.

```

26,700K: com.samsung.android.samsungpass (pid 26517)
33,621K: com.google.android.gms:snet (pid 26834)
31,321K: com.sec.android.app.samsungapps (pid 25282)
22,750K: com.samsung.android.app.reminder (pid 26556)
20,511K: com.google.android.permissioncontroller (pid 27081)
17,884K: com.google.android.projection.gearhead:com (pid 4724)
16,998K: com.samsung.android.stickercenter (pid 26548)
15,715K: com.facebook.appmanager (pid 8748)
15,355K: com.samsung.android.areojs (pid 26431)
14,977K: com.sec.android.app.vv.vetga (pid 26884)
12,827K: com.android.settings.intelligence (pid 27290)
12,804K: com.android.chrome:sandboxed_process:org.chromium.content.app.SandboxedProcessService0:7 (pid 16747)
11,822K: com.samsung.android.lool (pid 26882)
10,424K: com.google.android.apps.turbo (pid 26475)
8,919K: com.samsung.storyservice (pid 26412)
8,625K: com.samsung.android.intelligence (pid 27297)
8,525K: com.samsung.android.us.devicesecurity (pid 25945)
8,312K: com.samsung.faceservice (pid 25177)
8,204K: com.samsung.SM (pid 28084)
7,528K: com.sec.android.provider.badge (pid 6422)
6,243K: com.android.providers.calendar (pid 26578)
5,969K: com.samsung.android.privatespacestake (pid 26195)
5,912K: com.samsung.android.authfw (pid 26646)

Total PSS by category:
830,451K: Native
619,315K: Dalvik
579,626K: .apk mmap
215,408K: GL libtrack
209,159K: EGL libtrack
205,621K: dex mmap
152,554K: art mmap
152,932K: .so mmap
134,577K: Dalvik Other
87,794K: libarm
76,472K: .jar mmap
41,408K: stack
25,332K: Other mmap
13,301K: .oat mmap
15,632K: Other dev
7,614K: AShmem
1,088K: .ttf mmap
0K: Cursor
0K: Gfx dev
0K: Other wtrack

Total RAM: 7,615,488K (status normal)
Free RAM: 3,915,104K (1,877,424K cached pss + 1,534,228K cached kernel + 524,128K free)
PWA-BUF: 241,360K ( 209,160K mapped + 32,200K unmapped)
PWA-BUF: 241,360K
PWA-BUF Heaps pool: 136,456K
GPU: 341,460K ( 341,460K dmabuf + 0K private)
Used RAM: 6,524,160K (3,226,276K used pss + 997,884K kernel)
Lost RAM: 792,824K
DRAM: 615,156K physical used for 2,596,028K in smm (4,194,308K total smm)
Timing: 256 (large 512), com 322,560K, restore limit 187,520K (high-end-gfx)

```

Na captura obtida, observa-se que diversos processos do sistema e aplicativos de usuário estão em execução simultânea. Entre eles, destacam-se:

1. com.google.android.gms (serviços do Google),
2. com.facebook.appmanager,
3. com.samsung.android.lool (serviço interno da Samsung),
4. com.android.chrome (processo de navegador),
5. e com.samsung.android.privatespace.

Cada um apresenta seu respectivo PID (Process ID) e o valor de memória consumido, medido em KB

Esses dados representam o PSS (Proportional Set Size), que é a métrica usada pelo Android para calcular o quanto de memória física um processo realmente consome, levando em conta páginas de memória compartilhadas entre diferentes aplicações.

A seção “Total PSS by category” apresenta a distribuição da memória entre as principais áreas do sistema:

- Native (830 MB): memória utilizada por processos nativos do sistema e bibliotecas em C/C++.
- Dalvik (619 MB): memória usada pela máquina virtual responsável por executar aplicativos Android (baseados em Java).
- .apk mmap (579 MB): mapeamentos de arquivos APK carregados em memória.

Essas informações permitem compreender como a RAM é segmentada entre as partes do sistema operacional e os aplicativos do usuário.

No resumo final, o relatório indica:

- Total RAM: 7.615 MB (aproximadamente 7,6 GB de memória física total disponível);
- Free RAM: cerca de 3,9 GB livres, somando memória de cache e kernel;

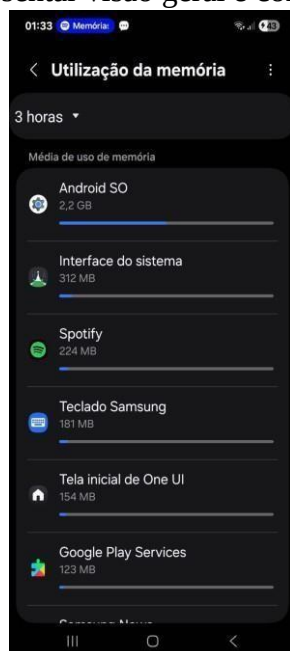
- DMA-BUF Heaps e GPU: mostram o uso de memória compartilhada entre CPU e GPU
(com 824 MB em buffers alocados para renderização gráfica);
- ZRAM: 615 MB de memória comprimida usada como swap, que auxilia a manter a fluidez do sistema em situações de alta demanda.

Os valores apresentados confirmam que o estado do sistema é normal, sem sinais de sobrecarga. A memória física está sendo utilizada de forma equilibrada, com ampla quantidade livre e mecanismos de compressão (ZRAM) atuando para reduzir a pressão sobre a RAM. Além disso, não há registro de falhas graves ou de falta de alocação, o que indica eficiência no gerenciamento de memória pelo kernel Android.

Em resumo, o `dumpsys meminfo` evidencia que o sistema Android realiza uma divisão estruturada de memória entre processos nativos, aplicações Dalvik/ART e componentes gráficos, mantendo boa performance e estabilidade mesmo com diversos aplicativos ativos em segundo plano.

Configurações → Apps → Informações de memória (Estatísticas de Processos)

- O que faz: interface do sistema mostrando uso médio de RAM por app e tempo de atividade.
- Como usar: abrir Configurações → Apps → selecionar aplicativo → memória/uso de memória.
- útil para apresentar visão geral e correlacionar com `dumpsys`.



O relatório de uso de memória evidencia que o sistema Android realiza uma gestão dinâmica e eficiente da memória RAM, priorizando processos do sistema e aplicativos em uso ativo. O maior consumo (2,2 GB) é atribuído ao próprio Android SO, o que é esperado, pois engloba o núcleo operacional e serviços essenciais. Aplicativos como Spotify e o teclado

Samsung apresentam consumo intermediário, refletindo o uso contínuo e a necessidade de resposta imediata.

Em síntese, os dados demonstram um uso otimizado da memória, com distribuição coerente entre sistema, interface e aplicativos, mantendo estabilidade e desempenho adequados ao perfil de uso do dispositivo.

3- ANÁLISE DOS COMANDOS E CONFIGURAÇÕES

3.1. Análise de uso da memória no Linux:

- Comandos para Monitoramento de Memória:

➡ vmstat:



Comando utilizado para exibir informações sobre a memória da máquina, tais como memória cache e buff.

```
labex:project/ $ vmstat
procs -----memory----- --swap--  -----io----- -system-- -----cpu-----
 r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs  us sy id wa st
  0   0       0 3036480 127036 6874916    0    0     7    69    0   0  4  1 95  0  0
```

➡ Assim como diversos comandos de monitoramento no terminal Linux, o vmstat apresenta diversas opções de comandos que complementam o próprio.



-d: Executando vmstat -d, será exibido para o usuário estatísticas do disco da máquina.

```
labex:project/ $ vmstat -d
disk- -----reads----- --writes----- --IO-----
      total merged sectors      ms  total merged sectors      ms  cur   sec
loop0  10083      0  381295    531   5034      0 1295872    6789     0     6
loop1      0      0      0      0      0      0      0      0     0     0
loop2      0      0      0      0      0      0      0      0     0     0
loop3      0      0      0      0      0      0      0      0     0     0
loop4      0      0      0      0      0      0      0      0     0     0
loop5      0      0      0      0      0      0      0      0     0     0
loop6      0      0      0      0      0      0      0      0     0     0
loop7      0      0      0      0      0      0      0      0     0     0
vda    349206 141326 22708306 212868 70320939 12370832 766445202 34314197     0 26600
vdb    22930798 5664 1720458953 16025136 256444938 70318161 16929410624 511508696     0 137288
labex:project/ $
```



-s: Será apresentado uma tabela com as estatísticas da memória e sobre tais eventos que podem ocorrer.

```

labex:project/ $ vmstat -s
16105756 K total memory
4851032 K used memory
6395736 K active memory
2527160 K inactive memory
4532808 K free memory
127092 K buffer memory
6594824 K swap cache
0 K total swap
0 K used swap
0 K free swap
459541861 non-nice user cpu ticks
3973950 nice user cpu ticks
135389873 system cpu ticks
12155401776 idle cpu ticks
1966739 IO-wait cpu ticks
0 IRQ cpu ticks
11244099 softirq cpu ticks
0 stolen cpu ticks
871803329 pages paged in
8848608471 pages paged out

```

➡ free:



Este comando mostra a quantidade de memória que está livre e está sendo utilizada na máquina do usuário, além de mostrar a área de swap, a memória compartilhada e os buffers que estão sendo utilizados pelo Kernel.

```

labex:project/ $ free
              total        used        free      shared  buff/cache   available
Mem:          16105756      5882324      3458456       120068      6764976      9766264
Swap:              0              0              0

```

➡ Como pode ser visto pela captura de tela acima, a área de swap não está sendo utilizada pela máquina no momento.



Agora, a explicação sobre cada um dos campos que estão sendo exibidos ao executar o comando free:

- ➡ total: Informa ao usuário o total de memória que há em sua máquina ➡
- used: Total de memória que está sendo utilizada pela CPU.
- ➡ free: Memória que está livre na máquina e pronta para ser utilizada.
- ➡ shared: Memória compartilhada, total de memória que poderá ser acessada por diversos programas.

➡ buff ou cache: É informado o quanto o usuário pode usar de uma memória buffer, uma área de armazenamento temporário de dados na memória RAM.

3.1.1. Análise de Memória do Processo:



Comando **htop** será utilizado para analisar e identificar processos que mais consomem memória:

0[|||||]8.8%Tasks: 33, 38 thr; 1 running

1[|||||]4.1%Load average: 0.19 0.33 0.43

2[|||||]4.1%Uptime: 371 days(!), 15:53:46

3[|||||]2.7%

Mem[|||||]4.52G/15.4G

Swp[|||||]0K/0K

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
36	labex	20	0	737M	101M	57656	S	6.1	0.6	0:15.84	/usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
203	labex	20	0	737M	101M	57656	S	1.4	0.6	0:03.95	/usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
204	labex	20	0	737M	101M	57656	S	1.4	0.6	0:03.95	/usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
205	labex	20	0	737M	101M	57656	S	1.4	0.6	0:03.91	/usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
1	root	20	0	11204	3800	3524	S	0.0	0.0	0:00.01	/bin/bash /etc/shiyanlou/sbin/init.sh
21	root	20	0	40824	28116	10672	S	0.0	0.2	0:00.53	/usr/bin/python3 /usr/bin/supervisord -n
22	root	20	0	15424	9120	7484	S	0.0	0.1	0:00.01	sshd: /usr/sbin/sshd -D [listener] 0 of 10-100
23	root	20	0	14040	4380	3836	S	0.0	0.0	0:00.00	su labex -c vncserver -fg -disableBasicAuth -pu
24	labex	20	0	40316	30564	6508	S	0.0	0.2	0:00.54	/usr/bin/perl /usr/bin/vncserver -fg -disableBa
39	labex	20	0	737M	101M	57656	S	0.0	0.6	0:00.00	/usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
40	labex	20	0	737M	101M	57656	S	0.0	0.6	0:00.14	/usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
46	labex	20	0	11204	3364	3116	S	0.0	0.0	0:00.00	sh -c { echo 'Running /home/labex/.vnc/xstartup
47	labex	20	0	11204	1916	1652	S	0.0	0.0	0:00.00	sh -c { echo 'Running /home/labex/.vnc/xstartup

F1HelpF2SetupF3SearchF4FilterF5TreeF6SortByF7NiceF8Nice +F9KillF10Quit

➡ Analisando a captura de tela, é mostrado que o **PID 36** é o processo que está mais consumindo da memória e fazendo a CPU consumir mais, com 6.1 e com um consumo de memória de 0.6% da máquina.

➡ Esse consumo de memória não é algo muito alto, mas irá depender da sua máquina e do processo que está sendo executado.

➡ E também como pode ser observado, há outros processos em execução que estão consumindo praticamente o mesmo número de memória que o processo com o PID apresentado.



Foco Concentual:

➡ **Memória Virtual(VIRT):** Representa o espaço total de deslocamento virtual para o processo. Inclui todas as páginas que um processo pode acessar, mesmo aquelas que atualmente não se encontram na RAM física.

↘ É o tamanho lógico da memória do processo. Pode ser até maior que a RAM disponível, pois inclui páginas armazenadas em disco.

➡ Por exemplo, um processo com 1 GB de memória virtual pode ter apenas 500 MB na RAM, com o resto sendo repaginado para disco.

→ **Memória Residente (RSS):** Mede a quantidade de memória virtual que está atualmente carregada na RAM física e o uso imediato para processo.

↘ Podemos dizer que é um subconjunto da memória virtual. Páginas residentes são aquelas ativadas com frequência. As não residentes estão em disco e precisam ser paginadas para a RAM quando necessárias.

→ Como exemplo, mesmo com 1GB de virtual, a RSS pode ser de 300 MB se o processo não estiver usando tudo ao mesmo tempo.

→ **Memória Compartilhada (SHR):** Parte da memória virtual que é compartilhada entre múltiplos processos, como bibliotecas dinâmicas ou regiões de memória mapeadas em um arquivo compartilhado

↘ Conta para o total da memória virtual de cada processo envolvido, mas não é duplicada na RAM, onde economiza espaço físico. É incluída no RSS se estiver carregada na memória.

→ Usando de exemplo dois processos usando a mesma biblioteca de 100 MB contam 100 MB cada em sua memória virtual, mas a RAM apenas aloca uma cópia física, economizando 100 MB.



Agora, para apresentar a diferença em cada um dos conceitos apresentados a cima, será utilizado três tópicos. **Escopo, Impacto no Desempenho e Contabilização:**

→ **Escopo:** A memória virtual é o total alocado (isso já incluindo o que está em disco), enquanto a residente é apenas o que está na RAM. A compartilhada é uma categoria dentro da virtual, focada na reutilização de processos.

→ **Impacto no Desempenho:** Alta memória virtual com baixo RSS indica ineficiência. Memória compartilhada alta mostra eficiência no processo, pois reduz duplicações. Diferenças grandes entre virtual e residente podem sinalizar problemas com thrashing.

→ **Contabilização:** Virtual e residente são medidas absolutas por processo; compartilhada é aditiva, mas não conta duplicadamente na RAM total do sistema.



Agora, como cada conceito irá refletir nas métricas do conceito de paginação do Sistema Operacional:

→ A Memória Virtual mostra o total de páginas alocadas, incluindo aquelas paginadas para o disco. Um valor alto indica potencial para swapping, onde páginas são trocadas para liberar RAM.

→ A Memória Residente trás as páginas ativamente na RAM, essenciais para a execução rápida. Baixo RSS em relação á virtual sugere que o processo está causando muitas faltas de página, forçando paginação e lentidão.

→ Memória Compartilhada irá otimizar a paginação ao reduzir a necessidade de duplicar páginas na RAM. Por exemplo, uma biblioteca compartilhada é paginada uma vez e reutilizada, diminuindo o tráfego de disco e melhorando o desempenho geral de processos.

```
top - 12:05:47 up 234 days, 8 min, 0 users, load average: 0.57, 0.28, 0.21
Tasks: 34 total, 1 running, 33 sleeping, 0 stopped, 0 zombie
%Cpu(s): 2.8 us, 0.8 sy, 0.0 ni, 96.4 id, 0.0 wa, 0.0 hi, 0.1 si, 0.0 st
MiB Mem : 15728.3 total, 5566.0 free, 3347.0 used, 6815.3 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 12004.0 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
36	labex	20	0	755572	103828	58036	S	0.7	0.6	0:13.06	Xvnc
1	root	20	0	11204	3892	3616	S	0.0	0.0	0:00.01	init.sh
21	root	20	0	40824	28196	10752	S	0.0	0.2	0:00.12	supervisord
22	root	20	0	15424	8880	7304	S	0.0	0.1	0:00.00	sshd
23	root	20	0	14040	4412	3868	S	0.0	0.0	0:00.00	su
24	labex	20	0	40284	30768	6756	S	0.0	0.2	0:00.38	vncserver
46	labex	20	0	11204	3528	3280	S	0.0	0.0	0:00.00	sh
47	labex	20	0	11204	1940	1676	S	0.0	0.0	0:00.00	sh
48	labex	20	0	454060	77164	61368	S	0.0	0.5	0:00.12	xfce4-session
57	labex	20	0	8300	2048	1592	S	0.0	0.0	0:00.00	dbus-launch
58	labex	20	0	8512	3340	2712	S	0.0	0.0	0:00.02	dbus-daemon
60	labex	20	0	309460	7648	6984	S	0.0	0.0	0:00.00	at-spi-bus-laun
65	labex	20	0	8424	4460	4004	S	0.0	0.0	0:00.00	dbus-daemon
69	labex	20	0	231000	6296	5492	S	0.0	0.0	0:00.03	xfconfd
75	labex	20	0	162748	8120	7352	S	0.0	0.1	0:00.00	at-spi2-registr
80	labex	20	0	7972	1076	0	S	0.0	0.0	0:00.00	ssh-agent



Observe com atenção na imagem acima o processo de **PID 60**, é possível notar que a Memória Residente (RES) está bem abaixo da Memória Virtual (VIRT), mostrando claro problema de page faults, onde muito provavelmente o processo está forçando a paginação dessas páginas que estão em falta, fazendo assim o processo ser executado de uma forma mais lenta. E mais, por causa do valor alto do virtual, o processo se torna um potencial para swapping.

3.1.2. Gestão da Memória Virtual:

- Agora será apresentado a forma em que o Sistema Operacional gerencia a troca de páginas, ou swapping;

→ O processo será dividido em quatro passos, sendo esses; **Alocação Inicial, Decisão de Swap Out, Processo de Swapping e Otimização.**

↘

A **Alocação Inicial** se trata do início do processo. o Sistema Operacional aloca páginas na RAM, se a memória RAM estiver cheia, ele identifica páginas vítimas para mover ao swap.

↘

Decisão de Swap Out acontece quando o Sistema Operacional usa algoritmos de substituição de páginas para escolher quais páginas remover da RAM. Exemplos comuns de algoritmos utilizados são:

↪ **LRU (Least Recently Used):** Remove páginas não acessadas recentemente, assumindo que o futuro se assemelha ao passado.

↪ **FIFO (First In, First Out):** Remove as páginas mais antigas da RAM.

↪ **Clock ou Second Chance:** Variantes otimizadas para eficiência.

➡ Lembrando que esses algoritmos são baseados em bits de referência, onde são marcados pelo hardware quando uma página é acessada, e ajudam a minimizar falhas de páginas.

↘

O **Processo de Swapping** é dividido em dois processos; Swap Out, quando uma página da RAM é copiada para o swap no disco, e sua entrada na tabela de páginas é atualizada para apontar para o disco. O processo de Swap In ocorre quando um processo tenta acessar uma página não presente na RAM, ocorre um page fault. O Sistema Operacional interrompe o processo, carrega a página de swap e retorna para a RAM, onde possivelmente irá remover uma outra página, e retorna a execução do processo.

→ O Sistema Operacional mantém uma tabela de páginas para rastrear locais, seja RAM ou disco, e irá usar o MMU para traduzir endereços virtuais em físicos.

↘

O processo final, a **Otimização**; o swap é configurado como um arquivo ou partição dedicada. O Sistema Operacional tenta minimizar swaps antecipando acessos e priorizando processos críticos.

↪ A **alta atividade de swap (thrashing)** ocorre quando o sistema passa mais tempo movendo páginas do que executando tarefas úteis, levando a degradação severa do desempenho. Vamos dividir os impactos em três partes;

➡ **Latência elevada da Memória:** Cada page fault causa pausas de segundos, isso em HDDs, ou milissegundos em SSDs, causando interrupção em processos. Como exemplo, um aplicativo pode parecer congelado durante swaps intensos.

➡ **Thrashing:** Quando a RAM é insuficiente para o conjunto de trabalho e processos ativos, sendo páginas frequentemente acessadas, o swap in ou out fica acessando repetidamente as mesmas páginas, criando um loop ineficiente.

➡ **Sobrecarga de recursos:** O processador e IO do disco ficam sobrecarregados, onde isso pode levar a afetar outros processos. Se for usado o exemplo de servidores, isso pode até mesmo causar falhas em aplicações extremamente críticas, enquanto em desktop leva a lentidão geral.

↪

Em resumo, o swap é essencial para estabilidade do processo, mas alta atividade sinaliza necessidade de utilizar mais da RAM, impactando diretamente a eficiência do sistema.

3.2. Análise de uso da memória no Windows:

Embora o Sistema Operacional Windows tenha passado por diversas atualizações ao longo do tempo, a forma como ele gerencia os diferentes tipos de memória também evoluiu significativamente. As melhorias contínuas no gerenciamento de processos e na utilização da memória tornaram o sistema mais estável e eficiente, principalmente com a introdução de novas tecnologias de otimização. Um exemplo marcante dessas evoluções é a memória cache, localizada dentro do processador, cuja função é acelerar o acesso aos dados mais utilizados, reduzindo o tempo de resposta da máquina. Nas versões mais recentes, como o Windows 11, a Microsoft implementou diversos aprimoramentos voltados para o desempenho geral do sistema, com destaque para o gerenciamento da memória virtual, responsável por ampliar a capacidade de processamento ao utilizar parte do disco rígido como extensão da memória RAM.

Atualmente, ao analisarmos o uso de memória no Sistema Operacional Windows, é possível identificar seis tipos principais de memória presentes na máquina:

- **Memória RAM:** utilizada para armazenar temporariamente dados e programas em execução;
- **Memória Secundária (Disco Rígido ou SSD):** responsável pelo armazenamento permanente de dados;
- **Memória Cache:** localizada no processador, otimiza a execução das tarefas e o acesso às informações;

- Memória Virtual: simula memória adicional no disco, usada quando a RAM está cheia;
- Memória ROM: contém informações essenciais gravadas pelo fabricante, que permanecem mesmo com o computador desligado;
- Memória Flash: empregada em dispositivos como SSDs, pendrives e smartphones, armazenando dados de forma não volátil.

Cada tipo de memória exerce um papel fundamental no funcionamento do sistema, contribuindo para o equilíbrio entre velocidade, armazenamento e estabilidade. Por exemplo, a memória ROM mantém informações mesmo após o desligamento do computador, garantindo que os dados essenciais de inicialização permaneçam preservados. Assim Para realizar a análise e o monitoramento de memória, o Windows disponibiliza diversas ferramentas nativas. O Gerenciador de Tarefas é a mais acessível e popular, permitindo ao usuário visualizar, em tempo real, a quantidade de memória sendo utilizada por cada processo. Outra ferramenta importante é o Monitor de Recursos (Resource Monitor), que fornece dados detalhados sobre o uso da memória física e virtual, além de possibilitar a observação de quais processos estão consumindo mais recursos. Já o Performance Monitor (perfmon) é uma ferramenta mais avançada, voltada para administradores e técnicos, capaz de gerar relatórios e gráficos personalizados sobre o desempenho do sistema. Além disso, o Windows oferece comandos como systeminfo e tasklist, que permitem obter informações detalhadas diretamente pelo Prompt de Comando ou PowerShell.

Diagrama hierárquico e tabelas para melhor entendimento:

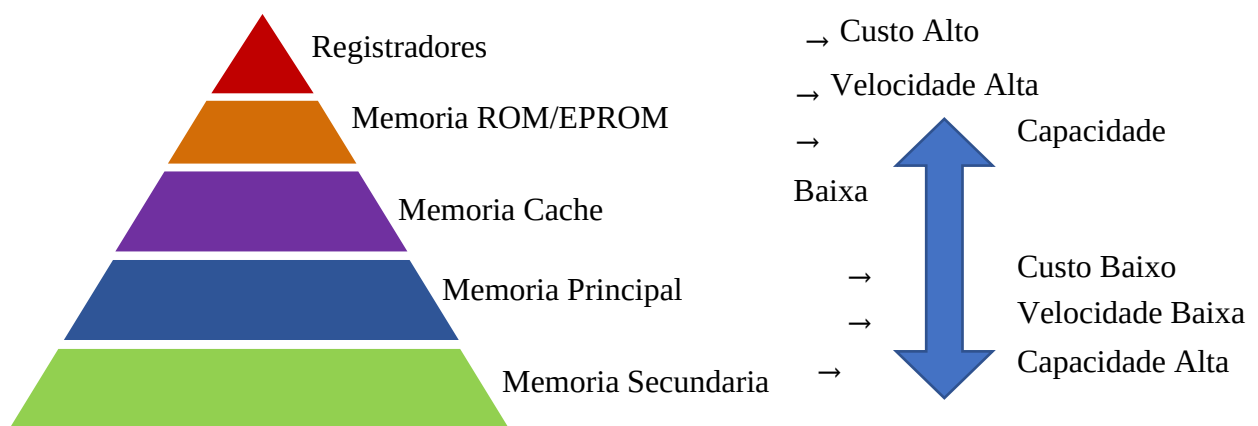


Tabela:

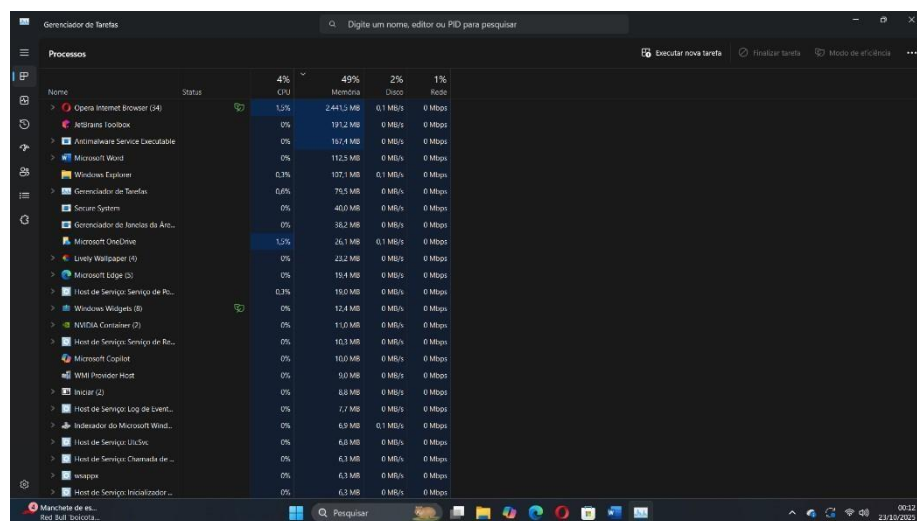
Tipo de memória	Função Principal	Volátil?	Velocidade	Capacidade	Localização
Cache	Acelerar a CPU	Sim	Muito Alta	Baixa	Processador
RAM	Armazenamento Temporário	Sim	Alta	Media	Placa-mãe

Virtual	Expansão de RAM	Não	Media	Alta	Disco Rígido
ROM	Armazenar BIOS	Não	Baixa	Baixa	Placa-mãe
Flash	Armazenamento Portatil	Não	Media	Variável	Pen-drive/SSD

3.2.1. Monitoramento da Memória do Sistema:

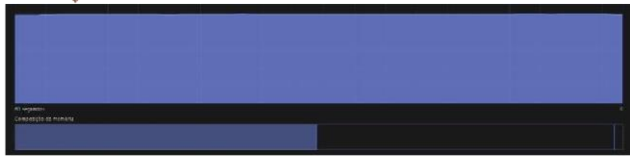
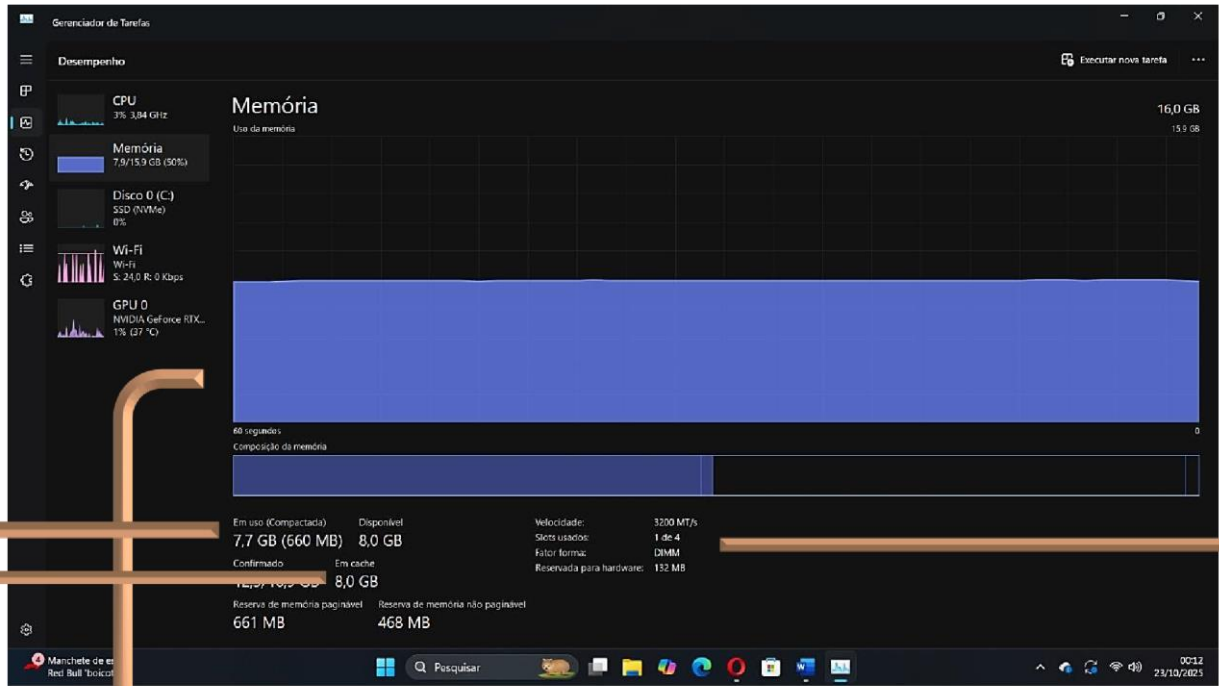
Para o monitoramento da memória pode-se usar variadas ferramentas dentro do Windows, uma delas e o mais comum, gerenciador de tarefas ele é o mais comum entre as ferramentas, pois com ele é possível monitorar, diagnosticar e controlar processos, serviços e o uso de recursos do sistema (CPU, memória, disco, rede e GPU). É a primeira linha de ação quando o sistema está lento ou um programa travou. Logo abaixo será exibido demonstrações das seguintes ferramentas e suas funções:

Gerenciador de tarefas:



Ao abrir a ferramenta na primeira guia que se estabelece ao usuário será exibido os seguintes processos sendo realizados. Assim temos uma informação mais detalhada sobre os programas que estão consumindo normal ou de maneira exagerada a CPU, RAM. Disco Rígido e a rede. Com base nisso também se e percebido variadas opções sobre

gerenciar os processos como pausar e encerrar, pois se identificarmos um programa nocivo usando a CPU podemos interromper A tarefa e então será mais executado.



Mostra um gráfico dinâmico que representa a quantidade de memória usada (em MB ou GB) e a quantidade livre com a cor azul. Indicando o uso atual da RAM e por cima mostra o comportamento ao longo do tempo (últimos 60 segundos).

Em uso (Compactada) Disponível
7,7 GB (660 MB) 8,0 GB

Mostra quanto da RAM está sendo usada ativamente por aplicativos, serviços e o próprio sistema.

Velocidade: 3200 MT/s
Slots usados: 1 de 4
Fator forma: DIMM
Reservada para hardware: 132 MB

Em cache
8,0 GB

Representa dados armazenados temporariamente na memória para acesso mais rápido depois (sem precisar buscar do disco), dessa forma o Windows usa cache de disco para acelerar reabertura de arquivos e programas.

Velocidade: 3200 MT/s
Mostra a frequência da RAM, medida em MHz (ex.: "2667 MHz").

Exibe quantos dos slots de memória da DIMM: usada em placa de vídeo ocupados. Mostra o tipo físico de memória a dispositivos (como placa -mãe estão ocupados). Parte da RAM dedicada a dispositivos (como placa -mãe estão ocupados). É útil desktops integradas. Ainda essa para saber se ainda há -SODIMM: usada em parte não pode ser RAM.

Slots usados: 1 de 4 Fator forma: DIMM Reservada para hardware: 132 MB
661 MB 468 MB

É a parte da memória usada pelo kernel do Windows (núcleo do sistema) e drivers que pode ser movida (paginada) para o arquivo de paginação (pagefile.sys) no disco rígido, quando não está sendo usada ativamente. Por esse método é usada em respectivas funções como:

- Drivers de dispositivos
- Componentes do sistema operacional
- Alguns serviços do Windows

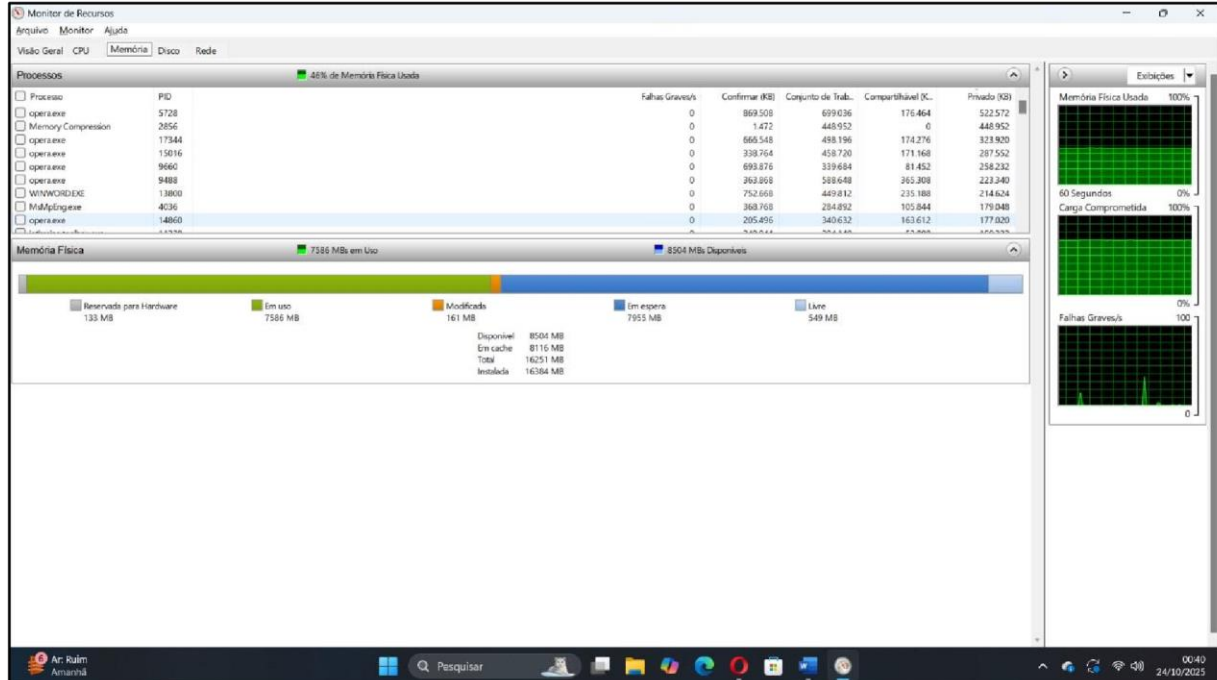
usada por programas.

É a parte da memória usada pelo sistema e drivers que nunca pode ser movida para o disco. Ela permanece sempre na RAM física, porque o sistema precisa acessar esses dados o tempo todo e com baixa latência. Ela é usada em essas funções:

- Drivers de hardware essenciais (placa de vídeo, rede, etc.)
- Estruturas de controle do kernel
- Rotinas críticas de interrupção

Após apresentar o gerenciador de tarefas e seus componentes informativos temos outras ferramentas como Resource Monitor (Monitor de Recursos) que será mostrado na página seguinte para uma melhor visualização:

Ao abrir o Resource Monitor e acessar a guia “Memória”, é possível obter uma **visão** aprofundada sobre o uso e a alocação da memória RAM no sistema. Assim diferente do Gerenciador de Tarefas, que exibe apenas o total utilizado e disponível, o Monitor de Recursos apresenta a distribuição completa da memória por categorias técnicas, permitindo compreender como o sistema operacional realmente gerencia o hardware de memória.



- **Hardware reserved:** Parte da memória física total (RAM) que é reservada pelo sistema para componentes de hardware.
- **In Use:** Refere-se à quantidade de memória ativamente utilizada por processos, drivers e pelo próprio sistema operacional no momento.
- **Modified:** Refere-se à memória que contém

O gráfico e tabela exibidos no Resource Monitor normalmente apresentam cinco principais seções: Hardware Reserved (Reservada pelo Hardware), In Use (Em uso), Modified (Modificada), Standby (Em espera) e Free (Livre),

dados alterados que ainda não foram gravados no disco. Ela não pode ser liberada até que o conteúdo seja salvo ou descartado, pois pode conter informações temporárias de programas e do sistema.

- **Standby:** Contém dados armazenados em cache que não estão sendo usados ativamente, mas podem ser reutilizados rapidamente se o mesmo programa for aberto novamente. Essa é uma das áreas mais importantes para o desempenho, pois reduz o tempo de carregamento.
- **Free:** Representa a quantidade de memória totalmente vazia, ou seja, que não contém dados e pode ser usada imediatamente por novos processos.

3.2.2. Pagefile (Arquivo de paginação/Swap):

Quando a máquina atinge o limite de utilização da memória RAM, torna-se necessário otimizar os processos ativos. Para isso, foi desenvolvida a área de Swap ou Pagefile, no caso do Windows, que funciona como uma extensão da memória física. Essa área utiliza uma porção do disco rígido (HD ou SSD) como memória virtual, permitindo que o sistema operacional armazene temporariamente processos ou páginas de memória que não estão sendo executados ativamente, mas que ainda ocupam espaço na RAM. Dessa forma, o sistema realoca essas

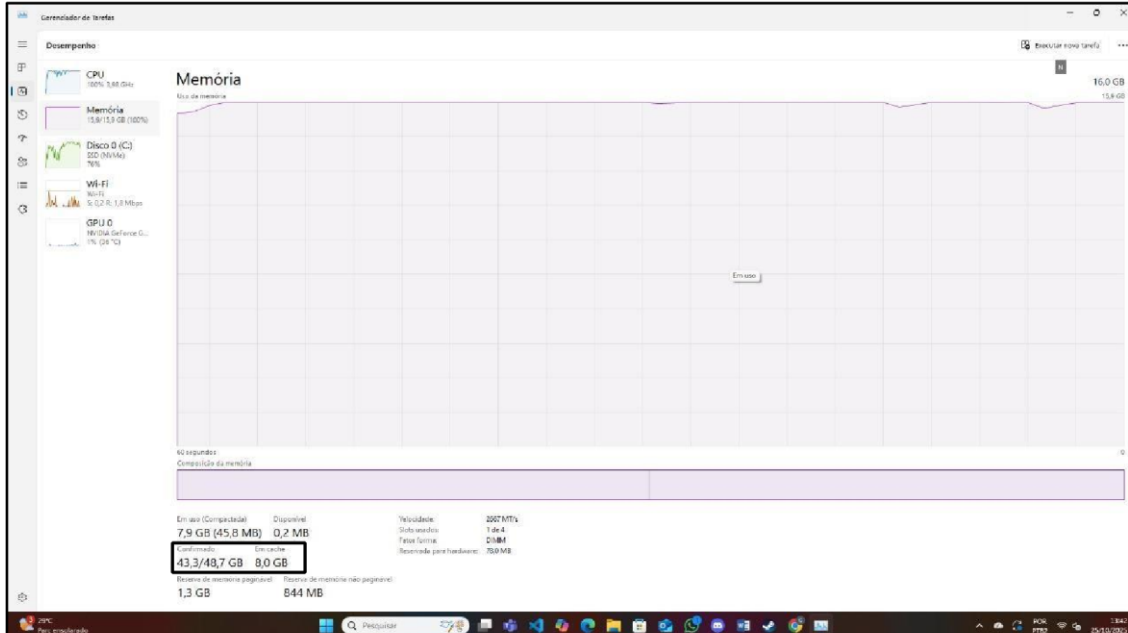
páginas menos utilizadas para o espaço de swap, liberando memória física para tarefas em execução e garantindo maior estabilidade ao sistema.

É importante destacar uma distinção terminológica: a área de Swap é o termo utilizado em sistemas baseados em Linux, enquanto o Pagefile é a denominação adotada pelo Windows. Uma diferença relevante entre ambos está na forma de criação e gerenciamento:

- No Windows, o gerenciador de memória configura e ajusta automaticamente o tamanho do arquivo pagefile.sys, embora o usuário possa definir manualmente o local e o tamanho do arquivo conforme suas necessidades.
- Já no Linux, a partição de swap é definida durante a instalação do sistema, podendo também ser criada ou ajustada posteriormente por meio dos comandos mkswap (para criação) e swapon (para ativação).

Assim, tanto o Swap quanto o Pagefile tem o papel essencial na gestão da memória virtual, assegurando o funcionamento estável do sistema mesmo em situações de alta demanda de memória física.

Uma captura de imagem será mostrada na página seguinte como exemplo de swap:



Nesta captura de tela exibimos a guia “memória” mostrando toda a RAM sendo utilizada. Logo embaixo visualizamos detalhes e se nota em “confirmados” o tanto que está usando da memória, “memória virtual”. (o usuário precisa se lembrar que tem de se a se restar o tanto em armazenamento físico da RAM) para saber o tanto que está sendo sendo utilizado de memória virtual.

Podemos usar outro método mais eficaz para saber o quanto está sendo usado de Memória virtual na máquina:

```
PS C:\Windows\system32> ^C
PS C:\Windows\system32> Get-CimInstance Win32_PageFileUsage
>>

Caption      Name          PeakUsage
-----
C:\pagefile.sys C:\pagefile.sys 0
```

Neste print se mostra o comando *Get-CimInstance win32_PageFileUsage* para o detalhamento do local, nome e o armazenamento que irá ocupar a memória virtual.

3.2.3. Análise de memória por processo:

Analisaremos processos dentro da memória e usaremos alguns comandos no power shell, e no task manager (gerenciador de tarefas) analisaremos as colunas de memória como:

- Working set (WS)

- Private Working Set

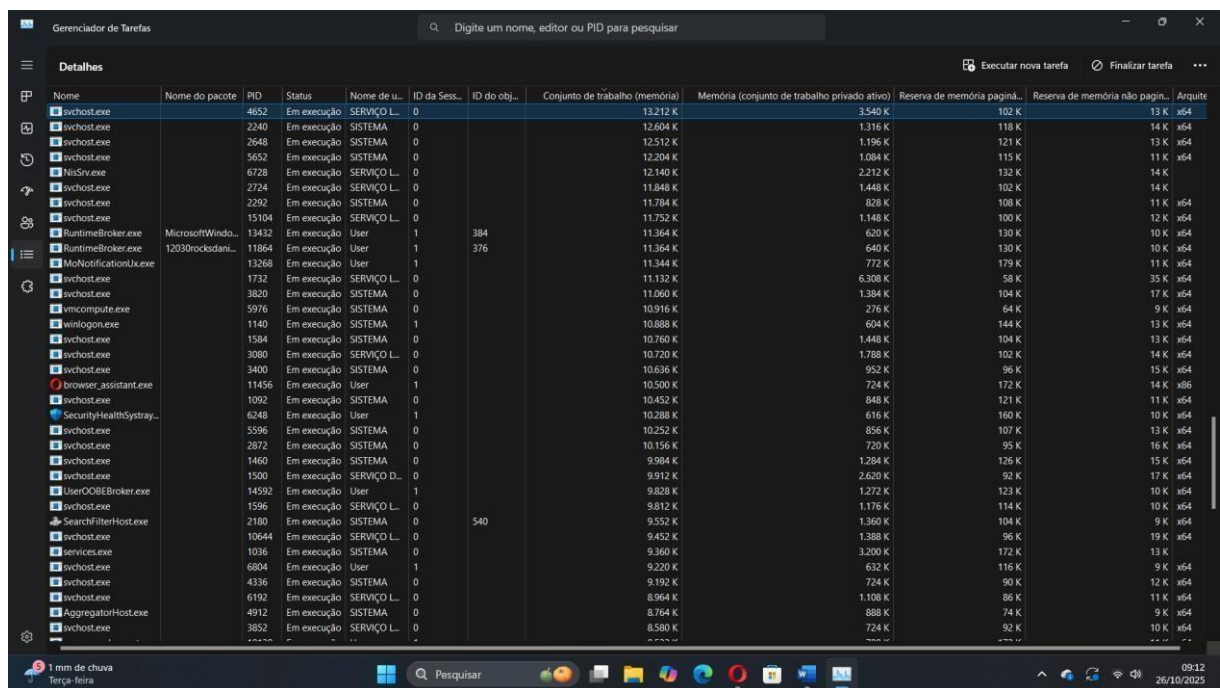
E no power shell usaremos comando para visualizar dados da memória por processo usando os seguintes comandos suas variedades de funções:

- Get process
- Sort object
- Property WS Decending

Working set (WS) e Private Working Set:

Essas colunas relevantes são gerenciadas pelo task manager assim exibindo várias métricas relacionadas ao uso da memória por cada processo assim representando diferentes formas de contabilizar a quantidade de memória em memória em uso. Como mostrado logo abaixo:

Quando se executado o gerenciador de tarefas, clicamos com o botão direito do mouse em cima qualquer coluna será exibido 2 opções “ocultar colunas” e “mostrar colunas”. Damos um click em mostrar colunas e selecionamos “conjunto de trabalho (memoria)” e logo em seguida “(memoria)conjunto de trabalho privado de trabalho” e pronto será exibidos as colunas que desejamos visualizar.



The screenshot shows the Windows Task Manager window with the 'Details' tab selected. The 'Conjunto de trabalho (memória)' column is right-clicked, and the context menu is open, showing the option 'Mostrar colunas' (Show columns) selected. The 'Mostrar colunas' submenu is also visible, showing the option 'Conjunto de trabalho (memória)' selected. The table below shows the memory usage for various processes.

Nome	Nome do pacote	PID	Status	Nome de u...	ID da Sess...	ID do obj...	Conjunto de trabalho (memória)	Memória (conjunto de trabalho privado ativo)	Reserva de memória página...	Reserva de memória não pagin...	Arquite
svchost.exe		4652	Em execução	SERVIÇO L...	0		13.212 K	3.540 K	102 K	13 K	x64
svchost.exe		2240	Em execução	SISTEMA	0		12.604 K	1.316 K	118 K	14 K	x64
svchost.exe		2648	Em execução	SISTEMA	0		12.512 K	1.196 K	121 K	13 K	x64
svchost.exe		5652	Em execução	SISTEMA	0		12.304 K	1.084 K	115 K	11 K	x64
svchost.exe		6728	Em execução	SERVIÇO L...	0		12.140 K	2.212 K	132 K	14 K	x64
svchost.exe		2724	Em execução	SERVIÇO L...	0		11.848 K	1.448 K	102 K	14 K	x64
svchost.exe		2292	Em execução	SISTEMA	0		11.784 K	828 K	108 K	11 K	x64
svchost.exe		15104	Em execução	SERVIÇO L...	0		11.752 K	1.148 K	100 K	12 K	x64
RuntimeBroker.exe	MicrosoftWind...	13432	Em execução	User	1	384	11.364 K	620 K	130 K	10 K	x64
RuntimeBroker.exe	12030rockdani...	11864	Em execução	User	1	376	11.364 K	640 K	130 K	10 K	x64
MoNotificationUx.exe		13268	Em execução	User	1		11.344 K	772 K	179 K	11 K	x64
svchost.exe		1732	Em execução	SERVIÇO L...	0		11.132 K	6.308 K	58 K	35 K	x64
svchost.exe		3820	Em execução	SISTEMA	0		11.060 K	1.384 K	104 K	17 K	x64
vmcompute.exe		5976	Em execução	SISTEMA	0		10.916 K	276 K	64 K	9 K	x64
winlogon.exe		1140	Em execução	SISTEMA	1		10.888 K	604 K	144 K	13 K	x64
svchost.exe		1584	Em execução	SISTEMA	0		10.760 K	1.448 K	104 K	13 K	x64
svchost.exe		3080	Em execução	SERVIÇO L...	0		10.720 K	1.788 K	102 K	14 K	x64
svchost.exe		3400	Em execução	SISTEMA	0		10.636 K	952 K	96 K	15 K	x64
browser_assistant.exe		11456	Em execução	User	1		10.500 K	724 K	172 K	14 K	x86
svchost.exe		1092	Em execução	SISTEMA	0		10.452 K	848 K	121 K	11 K	x64
SecurityHealthSystray...		6248	Em execução	User	1		10.208 K	616 K	160 K	10 K	x64
svchost.exe		5596	Em execução	SISTEMA	0		10.252 K	856 K	107 K	13 K	x64
svchost.exe		2872	Em execução	SISTEMA	0		10.156 K	720 K	95 K	16 K	x64
svchost.exe		1460	Em execução	SISTEMA	0		9.984 K	1.284 K	136 K	15 K	x64
svchost.exe		1500	Em execução	SERVIÇO D...	0		9.912 K	2.620 K	92 K	17 K	x64
UserOOBEBroker.exe		14592	Em execução	User	1		9.828 K	1.272 K	123 K	10 K	x64
svchost.exe		1596	Em execução	SERVIÇO L...	0		9.812 K	1.176 K	114 K	10 K	x64
SearchFilterHost.exe		2180	Em execução	SISTEMA	0	540	9.552 K	1.360 K	104 K	9 K	x64
svchost.exe		10644	Em execução	SERVIÇO L...	0		9.452 K	1.388 K	96 K	19 K	x64
services.exe		1036	Em execução	SISTEMA	0		9.360 K	3.200 K	172 K	13 K	x64
svchost.exe		6804	Em execução	User	1		9.220 K	632 K	116 K	9 K	x64
svchost.exe		4336	Em execução	SISTEMA	0		9.192 K	724 K	90 K	12 K	x64
svchost.exe		6192	Em execução	SERVIÇO L...	0		8.964 K	1.108 K	86 K	9 K	x64
AggregatorHost.exe		4912	Em execução	SISTEMA	0		8.764 K	888 K	74 K	9 K	x64
svchost.exe		3852	Em execução	SERVIÇO L...	0		8.580 K	724 K	92 K	10 K	x64

Conjunto de trabalho (memória)
13.212 K
12.604 K
12.512 K
12.204 K
12.140 K
11.848 K
11.784 K
11.752 K
11.364 K
11.364 K
11.344 K
11.132 K
11.060 K
10.916 K
10.888 K
10.760 K
10.720 K
10.636 K
10.500 K
10.452 K
10.288 K
10.252 K
10.156 K
9.984 K
9.912 K
9.828 K
9.812 K
9.552 K
9.452 K
9.360 K
9.220 K
9.192 K
8.964 K
8.764 K
8.580 K
8.532 K

A coluna do Working Set representa um meio de conjunto de páginas que um processo ativo está ocupando em um determinado momento. Outro dado interessante que ela pode se incluir com mais duas funções “memória privada” e a “memória compartilhada”, se em uma situação o sistema operacional pode modelar o tamanho do Working Set, mantendo mais paginas de RAM.

No Private Working Set indica a memória realmente dedicada a um processo, assim representado um processo está utilizando exclusivamente a memória RAM. Uma diferenciação e que ela não é compartilhada com nenhum outro processo, ela também não pode ser acessada por outros usuários.

Aqui embaixo será mostrado os funções do Working set.

Memória (conjunto de trabalho privado ativo)
3.540 K
1.316 K
1.196 K
1.084 K
2.212 K
1.448 K
828 K
1.148 K
620 K
640 K
772 K
6.308 K
1.384 K
276 K
604 K
1.448 K
1.788 K
952 K
724 K
848 K
616 K
856 K
720 K
1.284 K
2.620 K
1.272 K
1.176 K
1.360 K
1.388 K
3.200 K
632 K
724 K
1.108 K
888 K
724 K

-Uma curiosidade do Working Set é em caso de atingir o limite da memoria RAM, o SO pode transferir as paginas menos usada para o pagefile (Memoria Vitual).

Subcobjuntos ou

Gerenciador de Tarefas

Detalhes

Nome Nome do pacote PID Status Nome de usuário ID do processo ID do processo Memória (conjunto de trabalho privado ativo) Memória (conjunto de trabalho particular) Memória (conjunto de trabalho compartilhado)

Nome	Nome do pacote	PID	Status	Nome de usuário	ID do processo	ID do processo	Memória (conjunto de trabalho privado ativo)	Memória (conjunto de trabalho particular)	Memória (conjunto de trabalho compartilhado)
AggregatorHost.exe		4912	Em execução	SISTEMA	0		860 K	860 K	7.844 K
ApplicationHost.exe		8916	Em execução	User	1		2.228 K	2.228 K	27.788 K
AudioDg.exe		11476	Em execução	SERVICO L...	0		4.380 K	4.380 K	9.332 K
browser_assistant.exe		11328	Em execução	User	1		1.820 K	1.820 K	19.540 K
browser_assistant.exe		11456	Em execução	User	1		724 K	724 K	9.776 K
Copilot.exe		12736	Em execução	User	1	356	12.644 K	12.644 K	148.604 K
Copilot.exe		12692	Em execução	User	1	344	6.248 K	6.248 K	67.500 K
Copilot.exe		900	Em execução	SISTEMA	0		716 K	716 K	4.104 K
Copilot.exe		820	Em execução	SISTEMA	1		708 K	708 K	4.904 K
Copilot.exe		8956	Em execução	User	1		3.924 K	3.924 K	25.516 K
Copilot.exe		8116	Em execução	User	1		2.348 K	2.348 K	14.732 K
Copilot.exe		1844	Em execução	DWM-1	1		48.736 K	48.736 K	55.184 K
Copilot.exe		4428	Em execução	User	1		48.508 K	48.508 K	141.580 K
Copilot.exe		15952	Em execução	User	1		10.316 K	10.316 K	18.664 K
Copilot.exe		14644	Em execução	SISTEMA	0		2.272 K	2.272 K	18.980 K
Copilot.exe		1304	Em execução	UMFD-1	1		1.512 K	1.512 K	7.160 K
Copilot.exe		1312	Em execução	UMFD-0	0		16 K	16 K	2.716 K
Copilot.exe			Em execução	SISTEMA	0		288 K	288 K	
Copilot.exe			Em execução	User	1		0 K	0 K	
Copilot.exe		11944	Em execução	User	1		116.888 K	116.888 K	53.004 K
Copilot.exe		12572	Em execução	User	1	332	17.060 K	17.060 K	110.680 K
Copilot.exe		13200	Em execução	User	1	332	612 K	612 K	13.832 K
Copilot.exe		3784	Em execução	SISTEMA	0	68	612 K	612 K	3.416 K
Copilot.exe		1056	Em execução	SISTEMA	0		476 K	476 K	3.328 K
Copilot.exe		13268	Em execução	User	1		6.680 K	6.680 K	20.816 K
Copilot.exe		1796	Em execução	SISTEMA	0		752 K	752 K	10.572 K
Copilot.exe		4028	Em execução	SISTEMA	0	20	10.320 K	10.320 K	22.256 K
Copilot.exe		13232	Em execução	User	1	332	8.604 K	8.604 K	14.972 K
Copilot.exe		12156	Em execução	User	1		9.264 K	9.264 K	47.984 K
Copilot.exe		12220	Em execução	User	1		12.980 K	12.980 K	86.540 K
Copilot.exe		11520	Em execução	User	1	256	516 K	516 K	7.244 K
Copilot.exe		11532	Em execução	User	1	252	4.364 K	4.364 K	31.092 K
Copilot.exe		11596	Em execução	User	1	300	2.208 K	2.208 K	31.828 K
Copilot.exe		16880	Em execução	User	1	560	1.288 K	1.288 K	15.456 K
Copilot.exe			Em execução	User	1		180 K	180 K	124 K

Memória (conjunto de trabalho particular)

860 K
2.228 K
4.380 K
1.820 K
724 K
12.644 K
6.248 K
716 K
708 K
3.924 K
2.348 K
48.736 K
48.508 K
10.316 K
2.272 K
1.512 K
16 K
288 K
0 K
116.888 K
17.060 K
612 K
612 K
476 K
6.680 K
752 K
10.320 K
8.604 K
9.264 K
12.980 K
516 K
4.364 K
2.208 K
1.288 K
180 K

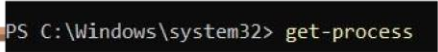
A memória privada representa a porção da memória física (RAM) usada exclusivamente por um processo no sistema operacional Windows, sem ser compartilhada com outros. Ela armazena dados e instruções essenciais ao funcionamento do programa, garantindo isolamento e segurança entre processo. Esse mecanismo assegura melhor desempenho e estabilidade ao sistema, além de proteger os dados contra acessos indevidos.

Memória (conjunto de trabalho compartilhado)

7.844 K
27.788 K
9.332 K
19.540 K
9.776 K
148.604 K
67.500 K
4.104 K
4.904 K
25.516 K
14.732 K
55.184 K
141.580 K
18.664 K
18.980 K
7.160 K
2.716 K
5.604 K
0 K
53.004 K
110.680 K
13.832 K
3.416 K
3.332 K
20.816 K
10.572 K
22.256 K
14.972 K
47.984 K
86.540 K
7.244 K
31.092 K
31.828 K
15.456 K
124 K

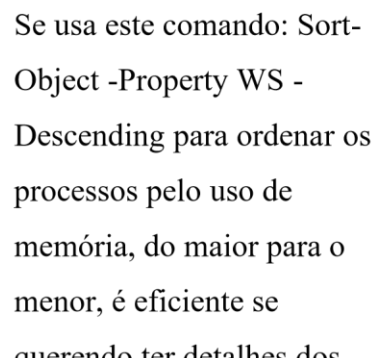
A memória compartilhada no Windows é uma área acessada por vários processos ao mesmo tempo, permitindo que eles utilizem os mesmos dados ou rotinas sem duplicá-los na RAM. Um exemplo são as bibliotecas DLL, que contêm funções usadas por diversos programas. Isso reduz o consumo de memória, melhora o desempenho do sistema e mantém a segurança, evitando que um processo altere dados de outro.

Logo após de o usuário visualizar as colunas Working set (WS) e Private Working Set, Podemos consultar as listas de processos com os comandos de uso de memória no power shell:



Handles: número de identificadores de recursos usados pelo processo.

- NPM(K): memória não paginada (em KB);
- PM(K): memória paginada;
- WS (K): Working set (memória física em uso);
- CPU: Tempo de CPU utilizado;
- Process Name: Nome do processo;



Se o usuário quiser saber de um programa em específico ele pode usar este código para visualizar os detalhes como mostrado abaixo:

- WS (Working Set): memória física usada pelo processo no momento;
- PrivateMemorySize: memória alocada apenas para o processo;
- PagedMemorySize: memória que pode ser movida para o disco;

```
Administrative: Windows PowerShell
150 11 1480 5086 0,00 2650 0 svchost
142 10 1308 5088 0,00 4896 0 svchost
138 8 1308 5086 0,00 3844 0 svchost
133 11 1220 5336 0,00 1540 0 svchost
767 45 118124 5342 8,78 16649 1 msedgebrowser
408 17 2738 9864 0,42 6161 1 localmount\localmountdaemon
908 24 2352 8776 1,16 980 0 csrss
68 6 826 2776 0,00 695 0 lsass
42 7 1692 2788 0,00 1312 0 fontdrvhost
5513 75 138024 1550 0,52 4884 2 SystemSettings
84 4 1172 1312 0,00 648 0 smss
552 31 143184 200 2,38 5888 1 msedgebrowser2
222 14 8572 96 8,40 7782 1 msedgebrowser2
181 17 9896 24 8,82 1676 1 msedgebrowser2
a w ldr x o ldr
PS C:\Windows\System32: Get-Process | Sort-Object WS -Descending | Select-Object opera, WS
opera WS
-----
505764896
49474784
49474784
94897168
39572800
385488032
38572800
522871296
38150880
29276160
79474784
24150880
268827184
222145184
216215152
262754880
166263936
188915840
188971184
18766784
165151568
176798616
172807512
164458880
162448912
161388892
173888112
155968848
```

```
PS C:\Windows\system32> Get-Process | Sort-Object WS -Descending | Select-Object opera, WS
```

```

PS C:\Windows\system32> Get-Process | Sort-Object PrivateMemorySize -Descending | Select-Object -First 10 ProcessName, PrivateMemorySize

ProcessName PrivateMemorySize
-----
opera 859037656
chrome 859037656
chrome.exe 715403126
chrome.exe 487441712
chrome.exe 38081254
chrome.exe 372989472
chrome.exe 351122312
chrome.exe 274308024
chrome.exe 268557968
chrome.exe 24778884

```

O comando `Get-Process | Sort-Object PrivateMemorySize -Descending | Select-Object -First 10 ProcessName, PrivateMemorySize` lista os 10 processos que mais consomem memória privada no sistema. A tabela que é mostrada identifica o nome do processo e a memória exclusiva usada, permitindo detalhar quais programas impactam mais o desempenho e ajudam no gerenciamento eficiente dos recursos do computador.

```

PS C:\Windows\system32> Get-Process | Sort-Object WS -Descending | Select-Object -First 10 ProcessName, WS, CPU, PrivateMemorySize, PagedMemorySize

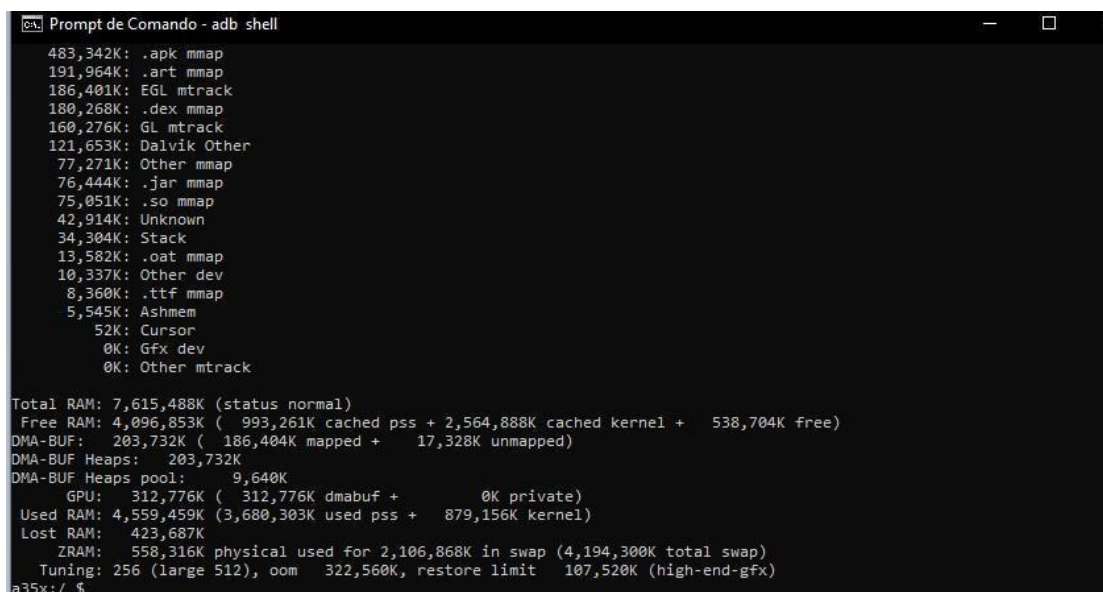
ProcessName WS CPU PrivateMemorySize PagedMemorySize
-----
opera 859037656 0.00% 859037656 859037656
chrome 859037656 0.00% 859037656 859037656
chrome.exe 715403126 0.00% 715403126 715403126
chrome.exe 487441712 0.00% 487441712 487441712
chrome.exe 38081254 0.00% 38081254 38081254
chrome.exe 372989472 0.00% 372989472 372989472
chrome.exe 351122312 0.00% 351122312 351122312
chrome.exe 274308024 0.00% 274308024 274308024
chrome.exe 268557968 0.00% 268557968 268557968
chrome.exe 24778884 0.00% 24778884 24778884

```

O comando do PowerShell apresentado permite identificar os 10 processos que mais consomem memória física, fornecendo informações essenciais para monitoramento de desempenho. A tabela resultante exibe o nome do processo, a memória em uso (WS), o tempo de CPU, a memória privada e a memória paginada, permitindo analisar quais aplicações demandam mais recursos do sistema e auxiliando no gerenciamento eficiente dos

3.3. Android:

Para o sistema Android, a análise de memória é feita via ADB (Android Debug Bridge), usando o comando `dumpsys meminfo`. A seção de interesse para o uso geral de memória é geralmente a que resume o uso total.



```
Prompt de Comando - adb shell
483,342K: .apk mmap
191,964K: .art mmap
186,401K: EGL mtrack
180,268K: .dex mmap
160,276K: GL mtrack
121,653K: Dalvik Other
77,271K: Other mmap
76,444K: .jar mmap
75,051K: .so mmap
42,914K: Unknown
34,304K: Stack
13,582K: .oat mmap
10,337K: Other dev
8,360K: .ttf mmap
5,545K: Ashmem
52K: Cursor
0K: Gfx dev
0K: Other mtrack

Total RAM: 7,615,488K (status normal)
Free RAM: 4,096,853K ( 993,261K cached pss + 2,564,888K cached kernel + 538,704K free)
DMA-BUF: 203,732K ( 186,404K mapped + 17,328K unmapped)
DMA-BUF Heaps: 203,732K
DMA-BUF Heaps pool: 9,640K
GPU: 312,776K ( 312,776K dmabuf + 0K private)
Used RAM: 4,559,459K (3,680,303K used pss + 879,156K kernel)
Lost RAM: 423,687K
ZRAM: 558,316K physical used for 2,106,868K in swap (4,194,300K total swap)
Tuning: 256 (large 512), oom 322,560K, restore limit 107,520K (high-end-gfx)
a35x:/ $
```

A saída do `dumpsys meminfo` (seção Total RAM) indica que o dispositivo possui 7,6 GB de RAM Total, dos quais 3,6 GB estavam em uso e 4,0 GB estavam livres.

Diferente do Windows e do Linux, o Android (que não utiliza um arquivo de swap tradicional) emprega uma abordagem mais agressiva de gerenciamento de memória, utilizando o kernel Linux com um LMK (Low Memory Killer) para encerrar processos em segundo plano (especialmente aqueles com o menor PSS – Proportional Set Size) quando a memória livre se torna criticamente baixa.

O fato de a RAM livre ser menor do que a usada é esperado em dispositivos móveis, pois o Android prefere manter a RAM preenchida com dados e aplicativos em cache para garantir uma retomada rápida. O sistema libera memória somente quando um aplicativo em primeiro plano (ativo) necessita de mais recursos. O campo Lost RAM estar zerado sugere que não houve problemas de alocação de memória para drivers ou componentes que não puderam ser rastreados.

Identificação de Vazamentos de Memória:

Sistema Windows (Análise com base nos dados fornecidos)

Para identificar possíveis vazamentos de memória e processos com consumo excessivo, foi utilizado o comando PowerShell que lista os cinco processos com maior uso de memória física (Working Set): PowerShell

```
PS C:\Users\shokt> Get-Process | Sort-Object WorkingSet -Descending | Select-Object -First 5
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
0	0	2964	799364		2040	0	Memory Compression
617	46	343632	413296	547,98	17728	1	opera
338	36	555288	410104	122,20	13208	1	RiotClientServices
568	53	331508	368108	407,86	13584	1	opera
617	43	373040	366180	341,88	8220	1	opera

A análise dos dados revela que o processo Memory Compression ocupa a maior porção de memória física, o que é um comportamento normal, pois o serviço compacta páginas de memória para otimizar o uso da RAM.

Além disso, múltiplas instâncias do navegador Opera.exe (consumo entre 354 MB e 371 MB) e o RiotClientServices.exe (cerca de 406 MB) são os maiores consumidores de RAM entre os aplicativos de usuário. O consumo elevado em navegadores é esperado, visto que criam processos separados para cada aba/extensão.

Durante o período de observação, não foram identificados vazamentos evidentes de memória. Os processos apresentaram comportamento condizente com suas funções. No entanto, o monitoramento contínuo dos processos do navegador é recomendado, pois o crescimento progressivo no valor de WS(K) sem novas abas abertas é o principal indicador de um vazamento de memória gradual (memory leak).

No Android, a identificação de consumo anormal se concentra no uso de Heap (memória alocada dinamicamente). Utilizamos o dumpsys meminfo com foco no resumo do aplicativo em questão.

```
** MEMINFO in pid 9308 [com.spotify.music] **
```

	Pss	Private	Private	SwapPss	Rss	Heap	Heap	Heap
	Total	Dirty	Clean	Dirty	Total	Size	Alloc	Free
Native Heap	80847	80332	508	39883	82308	177584	109493	62609
Dalvik Heap	29431	27348	1996	2611	33512	77130	28010	49120
Dalvik Other	9681	8568	56	136	11272			
Stack	2636	2632	4	2204	2644			
Ashmem	299	36	0	0	1100			
Other dev	16	0	16	0	400			
.so mmap	1035	144	4	56	27432			
.jar mmap	946	0	64	0	33956			
.apk mmap	52125	648	50372	1788	59920			
.ttf mmap	9	0	0	0	56			
.dex mmap	7	0	0	4	700			
.oat mmap	299	0	0	0	11328			
.art mmap	10999	8908	1676	1052	32184			
Other mmap	7424	64	7336	0	8464			
EGL mtrack	3752	3752	0	0	3752			
GL mtrack	18104	18104	0	0	18104			
Unknown	635	576	56	1001	1232			
TOTAL	266980	151112	62088	48735	328364	254714	137503	111729

O aplicativo em análise (com.spotify.music) apresentou um Total PSS (Proportional Set Size, medida precisa de impacto na RAM) de 266.980 KB (aproximadamente 267 MB). Deste total, o Native Heap é o maior consumidor (80.847 KB), seguido pelo Dalvik Heap (29.431 KB).

O Native Heap representa a memória alocada por código C/C++ (incluindo renderização de gráficos, que também aparece em GL mtrack: 18.104 KB e EGL mtrack: 3.752 KB), enquanto o Dalvik Heap é para objetos da linguagem Kotlin/Java.

Para detectar um vazamento de memória no Android, o foco deve estar no crescimento progressivo do Dalvik Heap ou Native Heap após ações repetidas do usuário (como abrir e fechar uma tela) sem que o uso de memória volte ao seu valor inicial. Um exemplo seria o Native Heap crescendo de 80.847 KB para um valor maior após o teste, sem liberação aparente, o que indicaria que objetos ou referências não estão sendo desalocados corretamente, sugerindo um vazamento de memória. Na análise pontual, o uso se manteve estável para as funções do aplicativo.

4- COMPARAÇÃO E ANALISE CRITICA:

Ferramentas:		Métricas	Descrição (de métricas)
Linux:	Top, Htop, Os.	VIRT, RES, PSS.	-VIRT: Quantidade total de memória virtual usada pelo processo (inclui bibliotecas e memória reservada) RES: Quantidade de memória física (RAM) realmente em uso. -PSS: Memória física usada pelo processo, dividindo proporcionalmente as bibliotecas compartilhadas entre processos.
Windows:	Gerenciador de tarefas, diagnostico de memória do Windows.	Working set (RES), Private bytes (PSS), virtual size (VIRT).	-Working set: Mostra a quantidade de RAM usada atualmente por um processo. -Private bytes: Memória alocada que não é compartilhada com outros processos -Virtual size: Quantidade total de espaço de endereçamento reservado, incluindo memória não usada.

Android:	DB shell, Top.	PSS; privated dirty, shared dirty.	-Roid ; mostra a memória proporcional usada pelo app. -Privated dirty : Memória usada que será liberada se o processo for encerrado. Shared dirty : Memória compartilhada com outros apps.
----------	----------------	------------------------------------	---

4.1. Discussão:

O modo em que a memória é gerenciada muda bastante entre os sistemas Windows e Linux e o sistema Android, principalmente por conta de suas diferenças de hardware e das prioridades de cada um. Nos computadores, o foco é demonstrar um bom desempenho para o usuário. O Linux e o Windows geralmente não costumam liberar grandes quantidades de memória rapidamente, pois preferem manter dados em cache pra acelerar a abertura de programas e o desempenho geral. Quando a RAM começa a encher, eles usam o Swap pra manter mais programas rodando “ao mesmo tempo”. Como os computadores normalmente têm mais memória física disponível se comparado com dispositivos móveis, o sistema não vê a necessidade de ficar fechando os aplicativos abertos a todo momento.

Já no caso do Android, o cenário muda. Sendo um dispositivo com menos memória e bateria limitada, o sistema precisa ser bem econômico. Quando a memória começa a ficar cheia, o Android suspende ou encerra aplicativos em que estão rodando de fundo automaticamente pra liberar espaço e garantir que o aplicativo em uso continue fluindo bem. Além disso, ele utiliza uma técnica que é denominada de “ZRAM”, que cria uma área de memória comprimida dentro da própria RAM (sendo basicamente um “winrar” para a memória). Isso permite guardar mais informações sem precisar usar o armazenamento interno (que é mais lento e gasta mais energia), mas exige um pouco mais de processamento pra comprimir e descomprimir os dados.

5. CONCLUSÃO:

Após a análise sobre como funciona o gerenciamento de memória de cada sistema, e possível notar a forma como cada um opera. Demonstrando os seus pontos distintos.

O Linux pode ser destacado pela eficiência e a flexibilidade, usando a memória virtual e cache de discos, ele garante o acesso rápido a informações. O sistema permite um controle com mais detalhes por parte do usuário e do administrador, oferecendo comandos e ferramentas que facilitam o monitoramento e o ajuste para a locação de memória dependendo da necessidade.

No caso do Windows, se pode notar que o sistema adota uma abordagem mais automatizada para facilitar o uso. Seu gerenciamento de memória da maior prioridade em questões de equilíbrio no seu desempenho e na estabilidade, utilizando métodos como

paginação e segmentação para entregar recursos de forma eficiente entre os processos. O sistema também possui um gerenciamento de memória dinâmico, além de um melhor entendimento sobre as páginas, o que ajuda no desempenho mesmo durante ações que exigem mais da máquina.

Já o Android, que embora baseado no kernel Linux, apresenta um gerenciamento de memória adaptado ao ambiente móvel. Ele prioriza a economia de recursos e a eficiência energética, já que precisa lidar com problemas como os limites do hardware e a bateria. O sistema faz o encerramento de maneira automática dos processos em segundo plano para liberar memória, além de adotar o conceito de low memory killer, que remove aplicações inativas para evitar lentidão ou travamentos. No Geral, o Android mantém um equilíbrio entre desempenho e consumo, que faz com que o usuário tenha uma boa experiência ao usuário final. De maneira pratica e otimizada, e possível destacar o uso consciente de alguns processos em segundo plano, a configuração adequada de swap (no caso do Linux), e a manutenção do sistema atualizado, que garante melhor gestão de recursos e maior compatibilidade com novas tecnologias de memória. Também é importante monitorar de maneira constante o uso de memória e evitar a sobrecarga de programas simultâneos, para que assim não ocorra casos de falha de desempenho na máquina.

Com a união dos dados analisados, e notável a diferença entre os sistemas quando se trata de gerenciamento de memória. O relatório mostra que foi possível entender e dominar os principais conceitos dos recursos de memória nos três sistemas. A comparação entre Linux, Windows e Android permitiu perceber a importância do gerenciamento eficiente no desempenho geral do sistema e para a estabilidade das aplicações. Dessa maneira e possível concluir o entendimento sobre a hierarquia da memória e o seu desempenho em diferentes sistemas operacionais.

6. AUTOAVALIAÇÃO:

Durante o processo de realização deste relatório, o grupo não enfrentou problemas para realizar suas partes, novamente mostrando organização e comprometimento de todos os integrantes, e que, após a realização de vários outros relatórios, todos conseguiram desempenhar suas funções da melhor maneira possível. Destacando algumas contribuições de cada integrante, o membro 1 ajudou desenvolvendo a introdução, junto com a organização das referências usadas para o relatório. O membro 2 ficou com a metodologia e a análise de comandos, além de organizar e enviar o relatório. Já os membros 3,4,5 ficaram responsáveis pela explicação do uso da memória no Windows, Linux e Android respectivamente, fazendo as capturas de tela e as tabelas. E o membro 6, responsável pela comparação crítica e a conclusão.

Todos os integrantes fizeram a autoavaliação de maneira conjunta. De forma geral, o grupo conseguiu entender como o gerenciamento de memória funciona e suas operações em cada sistema.

7. REFERENCIAS:

ALVES, Jósis. Sistemas Operacionais - Gerência de Memória. Disponível em: <<https://blog.grancursosonline.com.br/sistemas-operacionais-gerencia-de-memoria/amp>>. Acesso em: 26 oct. 2025.

Considerações de uso de memória no ajuste de desempenho do AD DS. Disponível em: <<https://learn.microsoft.com/pt-br/windowsserver/administration/performance-tuning/role/active-directory-server/memory-usage-considerations>>. Acesso em: 26 oct. 2025.

KAUSHIKA-MSFT. Introdução ao arquivo de paginação - Windows Client. Disponível em: <<https://learn.microsoft.com/pt-br/troubleshoot/windows-client/performance/introduction-to-the-page-file>>. Acesso em: 26 oct. 2025.

MARIANO, Marcio. Comandos Linux - Comando free. Linux Force Security, 4 Mar. 2025. Disponível em: <<https://linuxforce.com.br/comandos-linux/comandos-linux-comando-free>>. Acesso em: 26 oct. 2025

MUNDOTECNAUTA. Memória virtual no Linux 1/2. Disponível em: <<https://www.youtube.com/watch?v=Po6PvHcRvAY&t=15s>>. Acesso em: 26 oct. 2025.

O que é estouro de buffer? Ataques, tipos e vulnerabilidades. Disponível em: <<https://www.fortinet.com/br/resources/cyberglossary/buffer-overflow>>. Acesso em: 26 oct. 2025.

Power9 Reference Codes. Disponível em: <https://www.ibm.com/docs/ptbr/power9/0000REF?topic=POWER9_REF/p9hcg/p9hcg_vmstat.html>. Acesso em: 26 oct. 2025.

SHELDON, Robert. Pagefile. Disponível em: <<https://www.techtarget.com/whatis/definition/pagefile>>. Acesso em: 26 oct. 2025.

SIVAVISHNU. PSS: The definitive metric for Android memory footprint . Disponível em:

<<https://medium.com/@sivavishnu0705/pss-the-definitive-metric-for-androidmemoryfootprint-e5d1456c3603>>. Acesso em: 26 oct. 2025.

STEVENS, Peter. Understanding memory management in Windows 10 vs. Windows

11. Almaden.ai - Reimagined Digital Experience, 9 Oct. 2024. Disponível em:

<<https://almaden.ai/it-asset-management/memory-management-windows-11/>>. Acesso em: 26 oct. 2025

Uso do disco, melhorias na memória e desempenho de aplicativos do Windows - Windows apps. Disponível em:

<<https://learn.microsoft.com/pt-br/windows/apps/performance/diskmemory>>. Acesso em: 26 oct. 2025.

Visão geral do gerenciamento de memória. Disponível em:

<<https://developer.android.com/topic/performance/memory-overview?hl=pt-br>>. Acesso em: 26 oct. 2025.

WINDOWS-DRIVER-CONTENT. Resultados para a avaliação de Volume de Memória.

Disponível em: <[https://learn.microsoft.com/pt-](https://learn.microsoft.com/pt-br/windowshardware/test/assessments/resultsfor-the-memory-footprint-assessment)

[br/windowshardware/test/assessments/resultsfor-the-memory-footprint-assessment](https://learn.microsoft.com/pt-br/windowshardware/test/assessments/resultsfor-the-memory-footprint-assessment)>. Acesso em: 26 oct. 2025.

zram. Disponível em: <<https://wiki.archlinux.org/title/Zram>>. Acesso em: 26 oct. 2025.

Disponível em: <<https://www.baboo.com.br/base-de-conhecimento/arquivo-de-paginacaooupagefile>>. Acesso em: 26 oct. 2025a.

Disponível em: <<https://www.clubedohardware.com.br/forums/topic/1671877-o-queéarquivo-de-paginação>>. Acesso em: 26 oct. 2025b.

Disponível em: <[https://www.logicmonitor.com/blog/the-right-way-to-monitorvirtualmemory-on-linux#:~:text=into%20these%20concepts,-](https://www.logicmonitor.com/blog/the-right-way-to-monitorvirtualmemory-on-linux#:~:text=into%20these%20concepts,-,Key%20takeaways,efficiency%20and%20preventing%20performance%20bottlenecks)

[,Key%20takeaways,efficiency%20and%20preventing%20performance%20bottlenecks](https://www.logicmonitor.com/blog/the-right-way-to-monitorvirtualmemory-on-linux#:~:text=into%20these%20concepts,-,Key%20takeaways,efficiency%20and%20preventing%20performance%20bottlenecks)>.

Acesso em: 26 oct. 2025c.

Disponível em: <<https://askubuntu.com/questions/9642/how-can-i-monitor-the-memoryusage>>.

Acesso em: 26 oct. 2025d.

Disponível em:

<https://jvasconcellos.com.br/wpcontent/uploads/2011/04/ger_memo_swapping-2.pdf>.

Acesso em: 26 oct. 2025e.